

Implementing Neural Networks in numpy

Python implementation - numpy

To implement feedforward NNs, we will create a module that will include a few files with different components that will be complemented in the next session:

- *"neuralnet"* – it will include the main class to create a neural network that will "comunicate" with the other modules;
- "layers" – it will include the layers that will make the NNs (dense, etc);
- "activations" – it will include the activation layers (sigmoid, tahn, relu, etc);
- "data" – dataset (similar to previous ones)
- "optimizer" – training algorithms used to update the parameters of the NNs during training (SGD)
- "losses" – loss functions to optimize during training (CrossEntropy, MeanSquaredError, etc.)
- "metrics" – error metrics

Python implementation - Layers

- We will first implement a base layer with some shared and **abstract** methods for all other specific layers.
- class Layer:
 - methods:
 - forward_propagation – abstract method that computes the output of a layer given the input;
 - output_shape – abstract method that returns the output shape of the layer;
 - parameters – abstract method that returns the number of parameters of the layer;
 - set_input_shape – sets the input shape of the layer;
 - input_shape – returns the input_shape of the layer;
 - layer_name – returns the name of the layer;

Python implementation – Dense Layer

- A dense layer, also known as a fully connected layer, connects each neuron to every neuron in the previous and next layers, forming a dense network of connections.
- `class DenseLayer(Layer):`
 - arguments:
 - `n_units` – number of neurons in the NN;
 - `input_shape` – tuple with the input_shape (for tabular data it will be (n_features,))
 - estimated parameters:
 - `input` – the layer input;
 - `output` – the layer output;
 - `weights` – the layer weights;
 - `biases` – the layer biases;
 - methods:
 - `initialize` – initializes the layer with random weights and biases and with the optimizer;
 - `parameters` – returns the number of parameters of the layer;
 - **`forward_propagation` - perform forward propagation on a given input;**
 - `output_shape` - returns the shape of the output of the layer.

**Complete this
function !**

Python implementation – Activation layers

- `class ActivationLayer(Layer):`
 - methods:
 - `forward_propagation` - perform forward propagation on a given input by applying the activation function;
 - `backward_propagation` - perform backward propagation on the given output error by applying the derivative of the activation layer on the input and multiplying it with the output error;
 - `activation_function` – abstract method that computes the activation function on the given input (used on the forward propagation);
 - `derivative` – abstract method that computes the derivative of the activation function on the given input (used on the back propagation);
 - `output_shape` - returns the shape of the output of the layer (which is the same as the `input_shape`);
 - `parameters` – returns the number of parameters of the layer (returns 0 as activation layers do not have learnable parameters).

Python implementation – Activation layers

- class ReLUActivation(ActivationLayer):
 - methods:
 - activation_function – applies the ReLU function on the input (argument)
 - **derivative – applies the ReLU derivative on the input (argument)**
- class SigmoidActivation(ActivationLayer);
 - methods:
 - activation_function – applies the sigmoid function on the input (argument)
 - **derivative – applies the sigmoid derivative on the input (argument)**

Complete these functions !

Complete these functions !

Python implementation – Loss functions

- We will first implement a base loss function class with two abstract methods for all other specific layers (the class will not need an `__init__`).
- class `LossFunction`:
 - methods:
 - `loss` – abstract method that computes the loss based on the true and predicted labels
 - `derivative` - abstract method that computes the derivative of the loss based on the true and predicted labels
- class `MeanSquaredError(LossFunction)`:
 - `loss` - compute the mean squared error loss function;
 - `derivative` - compute the derivative of the mean squared error loss function
- class `BinaryCrossEntropy(LossFunction)`:
 - `loss` - compute the binary cross-entropy loss function;
 - **`derivative` - compute the derivative of the binary cross-entropy loss.**

Complete these functions !

Python implementation – neural network

- Now that we have all the components we can build a NN (forward computation). The NeuralNetwork class will connect all the other classes together.
- class NeuralNetwork:
 - estimated parameters:
 - layers – the layers of the NN.
 - methods:
 - add – add a layer to the NN.
 - **forward_propagation – forward propagation of the full feedforward NN, going through all layers to compute final output for a set of examples**
 - predict – make predictions on new data (calls forward propagation);

**Complete this
function !**

Python implementation – optimizer

- Optimizers are algorithms that adjust the model's parameters during training to minimize the loss function.
- We will implement the SGD optimization algorithm, with the option of considering the momentum
 - arguments:
 - learning_rate – the learning rate to use for updating the weights;
 - momentum – the momentum to use for updating the weights.
 - estimated_parameters:
 - retained_gradient – accumulated gradient from previous epochs;
 - methods:
 - update - Update the retained_gradient, computes and returns the updated weights;

Python implementation – Dense Layer

- A dense layer, also known as a fully connected layer, connects each neuron to every neuron in the previous and next layers, forming a dense network of connections.
- `class DenseLayer(Layer):`
 - arguments:
 - `n_units` – number of neurons in the NN;
 - `input_shape` – tuple with the input_shape (for tabular data it will be (n_features,))
 - estimated parameters:
 - `input` – the layer input;
 - `output` – the layer output;
 - `weights` – the layer weights;
 - `biases` – the layer biases;
 - methods:
 - `initialize` – initializes the layer with random weights and biases and with the optimizer;
 - `parameters` – returns the number of parameters of the layer;
 - `forward_propagation` - perform forward propagation on a given input;
 - **`backward_propagation` - perform backward propagation on the given output error;**
 - `output_shape` - returns the shape of the output of the layer.

**Complete this
function !**

Python implementation – neural network

- Now that we have all the components we can build a NN. The NeuralNetwork class will connect all the other classes together.
- class NeuralNetwork:
 - arguments:
 - epochs - the number of epochs to train the neural network;
 - batch_size - the batch size to use for training the neural network;
 - optimizer - the optimizer to use for training the neural network;
 - learning_rate - the learning rate to use for training the neural network;
 - momentum – parameter of momentum;
 - verbose - whether to print the loss and the metric at each epoch;
 - loss - the loss function to use for training the neural network;
 - metric - the metric to use for training the neural network;

Python implementation – neural network

- class NeuralNetwork:
 - estimated parameters:
 - layers – the layers of the NN.
 - history – dictionary containing the loss and metric at each epoch.
 - methods:
 - add – add a layer to the NN.
 - get_mini_batches – handle training batches yielding sets of examples for training
 - forward_propagation – forward propagation of the full feedforward NN, going through all layers to compute final output for a set of examples
 - **backward_propagation – backward propagation of the full feedforward NN for a set of examples, going through all layers in reverse order to calculate errors and update weights**
 - fit – train the NN (will call both forward and backward propagation).
 - predict – make predictions on new data (calls forward propagation);
 - score – evaluate the NN on a given dataset (calls predict);

**Complete this
function !**

Python implementation – neural network

NeuralNetwork.fit:

- arguments:
 - dataset – the dataset to train the NN on
- algorithm:
 1. Compute mini batches of size self.batch_size;
 2. Perform forward propagation for all layers;
 3. Compute the loss derivative error;
 4. Backpropagate the error through all layers using backward propagation;
 5. Repeat steps 2, 3 and 4 for all mini batches;
 6. Compute the loss based on true labels and predictions;
 7. Compute the metric based on true labels and predictions;
 8. Save the loss and metric in the history dictionary;
 9. Print the metric and loss if verbose is set to True;
 10. Repeat the previous steps for all epochs.

**Execute the
run.py script**